

CGA NLU Utilization Guide

Target: Bot/dialog design operators, AI/NLU experts

This document selects CGA's **ML**, **Semantic**, and **LLM** engines and organizes data, settings, testing, and quality improvement methods for each engine.

1. NLU basic concepts

- Intent: A classification unit that indicates what the user's utterance is requesting.
- Learning sentences: User expressions registered to learn intent or use as search criteria.
- Entity: The business value or name that must be extracted from the utterance.
- Dictionary: Assets for interpreting domain terms, synonyms, and user expressions
- Threshold: Minimum criteria to accept results as usable.
- Similarity: A value that indicates how close the input and candidate data are in meaning.
- Answer engine: Area to select a given answer or search/create answer based on classification results

2. Engine selection criteria

Category	ML	Semantic	L.L.M.
Basic method	Classification based on intent and learning sentences	Embedding and vector search focus	LLM model and directive-based processing
ready data	Intention, learning sentence, object, dictionary	Intent or search knowledge, embedding·Vector DB	Intent, Directive, Provider·Model
A good starting point	When there is a distinguishable work intention and management of learning sentences	When you need to search for similar meaning even though the expressions are different	When LLM-based analysis/generation and model operation are needed
Key Verification	Accuracy, misclassification, sentence balance	Search similarity, threshold, index status	Response consistency, compliance with instructions, delay/cost
Key risks	Intent duplication, data imbalance	Embedding compatibility, index mismatch	Model Change, Prompt Impact, Response Deviation

Detailed model and support status are checked based on the selections available on the bot creation screen.

3. Common operating procedures

1. Check the bot and version.
2. Check the NLU method and model.
3. Check the answer method.
4. Prepare for data or connection setup.
5. Executes training, index creation, and model application tasks.
6. Test representative utterances in the simulator.
7. Check the results in the analysis/evaluation/dialogue history.
8. Triage the cause, correct the data, and test again.

4. Common principles of quality inspection

- One training sentence contains one core intent.
- Similar intents include nouns, verbs, and situations that can be distinguished from each other.
- Check for balance to avoid overloading data for specific intents.
- First determine whether domain terms will be managed as objects or dictionaries.
- Test utterances do not only repeat the same expressions as training sentences.
- First improve intent pairs with repeated misclassifications in the analysis results.

4.2 Rules for writing learning sentences

Note When applying the principles of the NLU Guide to CGA operational data, use the following sequence:

1. Name the intent to indicate the purpose of the task.
2. Only one core request is included in one training sentence.
3. Prepare the same intention with different word order, tone, and expression.
4. Don't just multiply words that overlap with other intents, include context and actions that distinguish the intent.
5. Do not register identical or nearly identical sentences repeatedly.
6. Compare each intent to ensure that sentences are not concentrated only with a specific intent.

For example, when dividing **Check remaining leave days** and **How to request leave**, prepare expressions that reveal the division criteria, such as **Days remaining** and **Application procedure**, rather than including the word **Leave** in both intentions.

4.3 QA/Knowledge Data Creation Rules

When preparing QA or document-based knowledge, be clear about the scope of the questions and answers.

- Questions are written in expressions that actual users can enter.
- Answers should be written in direct response to the question, and do not mix multiple topics in one answer.
- For document-based data, the original text structure is organized so that the distinction between title and body is maintained.
- If the table or list is important, ensure that the meaning is maintained after conversion.
- When revising a document, check the application status to ensure that existing and new documents are not searched at the same time.

4.1 Operational inspection records

When changing engine settings or training data, record the following items.

- Bots and Versions
- NLU method, model, answer method before and after change
- Changed intent/object/dictionary/QA or connection settings
- Executed learning/indexing/apply operations and screen status
- Representative success and failure utterances
- Simulator/analysis/evaluation results before and after change

Without this record, it is difficult to separate the effects of engine changes from the effects of data changes.

5. Guide for each engine

- ML Engine Utilization
- Semantic Engine Utilization
- LLM Engine Utilization

6. Utilizing ML Engine

6.1 Settings

The items identified as ML models in the current screen are DeepLearning Lite, TF-IDF Linear, and Keyword Baseline. The actual learning connection status is checked together with the selectable status and version settings.

6.2 Writing data

- Prepare representative expressions and various expressions for each intention.
- Contain only one intent per statement.
- Intents that require distinction, such as method and error queries, do not mix expressions.

- Check the number of sentences for each intent and the diversity of expressions.

6.3 Testing and Improvements

1. Prepare representative utterances and boundary utterances respectively.
2. Check the results in the simulator.
3. Look for misclassification intent and repeated expressions in the analysis/evaluation screen.
4. Fix data to reveal differences between competing intents.
5. Train again and repeat the same test.

6.4 Precautions

We do not assume that quality will automatically improve by simply increasing the number of training sentences. Rather than simple suffix changes, we add key expressions that distinguish intent and various expressions from actual users.

Check before changing ML:

1. In one sentence, explain the criteria for distinguishing from competitive intent.
2. Check whether each intent has a representative expression, variant expression, or boundary expression.
3. Check that there are no learning sentences that mix multiple intentions in one sentence.
4. Retest whether successful utterances before the change are maintained after the change.

7. Utilizing Semantic Engine

7.1 Type

- **Semantic - Vector Worker:** Type using CGA Vector Worker basic model and Local Vector DB
- **Semantic - External Embedding:** Type connecting external embedding and Local Vector DB

7.2 Settings

When you select Semantic NLU, the Intent Vector DB connection settings are displayed. In the External Embedding type, search API address, API Key selection input, and Index name can be used. The default Vector Worker type resolves the default connection and Index name.

7.3 Model selection

The current code defines external embedding options such as **ko-sroberta** for Korean general documents, **multilingual-e5** for multilingual, tables, and formats, and **bge-m3** for long documents and terms and conditions. The actual operational selection is a joint check of document characteristics, operational connectivity, and embedding compatibility.

7.4 Testing and Improvements

1. Prepare representative questions and expressions.
2. Check whether intent or knowledge data is reflected in Vector DB.
3. In the simulator, expressions with the same meaning and expressions with different meanings are tested separately.
4. Check search results, similarity, threshold, and index status.
5. If the search does not match, check the combination of data, embedding, and index.

The actual indexing completion status and search results screen must be confirmed after browser/execution verification.

Check before changing semantics:

1. Verify that the embedding model and the data being searched are compatible.
2. Verify that the Index name and connection target match the current bot version.
3. If you use an external search API, check with your administrator for response specifications and authentication settings.
4. Results before the index is updated are not interpreted as quality of new data.

8. Utilizing the LLM engine

8.1 Settings

If you select LLM Engine, you can set the LLM Provider and detailed model for each provider. Provider choices confirmed on the screen are Gemini, ChatGPT, Claude, Groq, Cerebras, Mistral, Ollama, and OpenRouter. For example, if you select ChatGPT, you will see **GPT-4o mini (Default)** and **GPT-4o (High quality)**. The list of providers and models may vary depending on your operating settings, and if you use Ollama, you may see separate address entries.

8.2 Instructions and response methods

LLMs should review NLU model selection and answer method selection together.

- LLM Engine Answers: How LLM generates answers
- LLM Engine RAG Answer: How to use retrieved knowledge and LLM together
- Defined answers: How to use predefined answers

Directions clearly document tone, response format, and limitations. After changing a directive, we compare consistency and exception responses with the same test set.

8.3 Testing and Improvements

1. Prepare representative questions, ambiguous questions, and prohibited or exception questions.
2. Fix Provider and Detailed Model.
3. Repeat the same input to check response consistency.

4. Verify compliance with the directive and the basis for your response.
5. Logs delays, costs, and failure responses.

Check before LLM change:

1. Record the provider and detailed model before and after changes.
2. Compare using the same input/instruction/response method.
3. Check realism, format compliance, forbidden responses, and failure responses separately.
4. If delays or costs are significant, document them with quality results.

9. Analysis and quality improvement

The analysis screen checks the cumulative classification results and the steps applied. Classification steps currently displayed on the screen may include Exclusion/Ignore, Small Talk, Exacting Matching, Rule, ML, Semantics, LLM, etc.

Quality improvement sequence:

1. Collect failed utterances.
2. Check the classification steps actually applied.
3. Isolate which area the problem is: intent, search, directive, or response.
4. Fix minimum range of data.
5. Retest existing successful and failed utterances together.

9.1 Test set configuration

When changing the engine or modifying data, do not use only one test utterance.

test set	Purpose	Example standards
Representative speech	Check normal main usage path	Frequently used expressions
Modified utterance	Check processing for expression changes	Changes in word order, tone of voice, and spacing
boundary utterance	Ensure Distinction from Competitive Intent	Other requests with similar words
exception utterance	Check processing of unsupported/ambiguous requests	Questions that are not part of the intent
recursive utterance	Confirm maintenance of success results before change	Previously successful utterance

You must use the same set before and after the change to compare the effects of engine changes or data supplements. Test results are recorded along with the bot, version, engine, model, and response method.

10. Error response

Symptoms	Check first	Next Action
Not classified as expected intent	Bot·Version·Engine·Data Status	Review representative/boundary utterances and competitive intentions together
Semantic No results found	Vector DB, Index, Embedding Compatibility	Check connection/index/data reflection status
LLM answers waver	Provider, model, directive, input	Compare to the same test set and narrow down the directives
Study or readiness status not complete	Learning history, error messages, selection combinations	Record screen status and communicate to operations personnel

Cases seen in validation ML bots:

- Symptom: Learning request is registered in Queue but remains as **Untrained** after refresh
- Simulator results: **Unclassified intent, Intent classification cannot run because there are no training sentences.**
- Judgment: Successful saving of training sentences and completion of ML training are separate states, so classification quality is not evaluated before training is complete.
- Action: Check the completion status of the learning history, and if there is no history, send the bot, version, engine, and request time to the operation manager.

Do not adjust the status by directly modifying the DB or CLI. The bot, version, engine, error message, and occurrence time on the screen are recorded and delivered to the operation manager.

11. Training/indexing execution and state judgment

- Before execution, confirm bot UUID, version, language, engine, model, answer method, saved assets, and the baseline test set.
- ML training and Semantic indexing can take more than three minutes. A request message alone is not proof of success.
- Confirm the final state in training history and then run Bot Test. Treat missing history, an unchanged untrained state, an empty index, or a failed LLM call as a failure.

12. Score and cut-off operation

1. Record the current cut-off and similarity criteria.
2. Collect score distributions for correct, incorrect, boundary, and unsupported utterances.
3. Change one criterion at a time and rerun the same regression set.
4. Evaluate both false acceptance and false rejection; never optimize only the displayed percentage.

13. Multilingual NLU operation

CGA supports Korean, English, Simplified Chinese, Japanese, Vietnamese, French, and German for the UI and bot language. Keep identifiers and API contracts canonical while authoring utterances, messages, entities, and evaluation sets in the target language.

Message lookup uses the requested language first and the bot language as fallback. Validate morphology, spacing, honorifics, accents, and language-specific variants separately; do not rely on literal translation alone.

14. Controlled engine experiments

- Create a separate working version instead of overwriting the operating version.
- Fix the test set and change only one of data, threshold, model, provider, prompt, or index settings per experiment.
- Record accuracy, failure type, response consistency, latency, cost, and operational constraints.
- Promote only a version that meets the agreed criteria and retains previous successful cases.

15. Quality improvement example

For delivery inquiries, separate schedule inquiries from current-status inquiries, prepare representative, variant, boundary, exception, and regression utterances, verify order-number extraction, retrain or reindex, and compare all sets again. A lower error count is not enough if successful utterances lose score or switch intent.

16. Operational reflection checklist

- ☐ Bot UUID, version, language, engine, model, and answer method were recorded.
- ☐ Before/after results were compared with the same test set.
- ☐ Training or indexing success was confirmed in history.
- ☐ Scores, similar-intent conflicts, and regression utterances were reviewed.
- ☐ Target-language messages and entity extraction were verified.
- ☐ Real channel, conversation, API, and Queue histories were checked.

Related Documents

- [View all CGA manuals](#)
- [CGA Getting Started](#)
- [CGA User Manual](#)
- [Engine comparison table](#)